

Part 1: Host Environment Setup

Objective: Install the specific version of Vitis AI required for the DPU v3.3 IP core.

1. Install Docker Engine Run the following commands on your Ubuntu host machine to set up the container environment.

Bash

```
sudo apt-get update
sudo apt-get install -y docker.io
# Add current user to docker group to run without sudo
sudo usermod -aG docker $USER
# RESTART YOUR COMPUTER/VM NOW
```

2. Download the Vitis AI Docker Image We must use version 1.4 to match the FPGA IP core.

Bash

```
docker pull xilinx/vitis-ai-cpu:1.4.1.978
```

3. Clone the Vitis AI Repository This provides the launch scripts needed to start the container.

Bash

```
git clone -b v1.4 https://github.com/Xilinx/Vitis-AI.git
```

Part 2: Workspace Preparation

Objective: Create a clean directory structure that Docker can access.

1. Create Project Folder Create a folder named `FPGA_AI_Work` in your Home directory.

Bash

```
mkdir -p ~/FPGA_AI_Work/images
```

2. Copy Required Files Move your files into this folder so the structure looks exactly like this:

Plaintext

```
/home/user/FPGA_AI_Work/
├── final_terrain_model.h5    (Your original trained model)
└── images/                  (Folder containing 10-20 random .jpg
    images for calibration)
```

Part 3: The AI Translation Flow

Objective: Convert the heavy PC model into a quantized FPGA-ready format.

1. Launch the Environment Run this command from your terminal to enter the Docker container.

Bash

```
# Adjust path to where you cloned Vitis-AI
~/Vitis-AI/docker_run.sh xilinx/vitis-ai-cpu:1.4.1.978 -v
~/FPGA_AI_Work:/workspace
```

2. Activate TensorFlow Toolchain Inside the Docker terminal:

Bash

```
conda activate vitis-ai-tensorflow2
```

3. Create the Master Script Create a script to rebuild the model structure and transplant weights (bypassing version errors).

Bash

```
nano master_flow.py
```

Paste the following code into master_flow.py:

Python

```
import tensorflow as tf
from tensorflow.keras.applications import MobileNetV2
from tensorflow.keras.layers import Dense, GlobalAveragePooling2D, Dropout,
Input
from tensorflow.keras.models import Model
from tensorflow_model_optimization.quantization.keras import vitis_quantize
import os

# CONFIGURATION
BROKEN_MODEL = "final_terrain_model.h5"
CLEAN_MODEL = "clean_model.h5"
QUANTIZED_DIR = "quantized_model"
CALIB_DIR = "images"

print("🚀 STARTING AI TRANSLATION FLOW...")

# STEP 1: REBUILD ARCHITECTURE (TF 2.3 Native)
print("🏠 Rebuilding clean architecture...")
input_tensor = Input(shape=(224, 224, 3))
base_model = MobileNetV2(weights=None, include_top=False,
input_tensor=input_tensor)

x = base_model.output
x = GlobalAveragePooling2D()(x)
x = Dense(128, activation='relu')(x)
x = Dropout(0.4)(x)
predictions = Dense(3, activation='softmax')(x)

clean_model = Model(inputs=base_model.input, outputs=predictions)

# STEP 2: TRANSPLANT WEIGHTS
print(f"✍ Injecting weights from {BROKEN_MODEL}...")
try:
    clean_model.load_weights(BROKEN_MODEL)
```

```

    print("✅ Weights loaded successfully!")
except Exception as e:
    print(f"❌ Error loading weights: {e}")
    exit()

# STEP 3: QUANTIZATION
print("🔄 Starting Quantization...")
if not os.path.exists(CALIB_DIR):
    print(f"❌ Error: Calibration folder '{CALIB_DIR}' missing.")
    exit()

datagen = tf.keras.preprocessing.image.ImageDataGenerator(rescale=1./255)
calib_set = datagen.flow_from_directory(
    ".", classes=[CALIB_DIR],
    target_size=(224, 224), batch_size=10
)

quantizer = vitis_quantize.VitisQuantizer(clean_model)
quantized_model = quantizer.quantize_model(calib_dataset=calib_set)

# STEP 4: SAVE OUTPUT
if not os.path.exists(QUANTIZED_DIR):
    os.makedirs(QUANTIZED_DIR)
quantized_model.save(os.path.join(QUANTIZED_DIR, "quantized_model.h5"))
print("🎉 SUCCESS! Quantized model ready.")

```

4. Execute the Script

Bash
python master_flow.py

Expected Output: 🎉 SUCCESS! Quantized model ready.

Part 4: Final Compilation (Bitstream Generation)

Objective: Compile the Int8 model into DPU instructions (.xmodel).

1. Define Target Architecture Create a file named `arch.json` to specify the Zynq-7020 DPU configuration.

Bash
nano arch.json

Content:

JSON
{ "target": "DPUCZDX8G_ISA0_B512_MAX" }

2. Run the Compiler Execute the compilation command:

Bash
vai_c_tensorflow2 \

```
--model quantized_model/quantized_model.h5 \  
--arch arch.json \  
--output_dir build_output \  
--net_name active_suspension_model
```

Part 5: Final Deliverables

If the process is successful, you will have the following files ready for the robot:

1. **Software Brain:** build_output/active_suspension_model.xmodel
2. **Hardware Body:** design_1_wrapper.xsa (From Vivado)